

时光像素 · 面试备战手册

读完这份文档，你可以和面试官谈任何关于这个项目的细节。

一、项目身份证

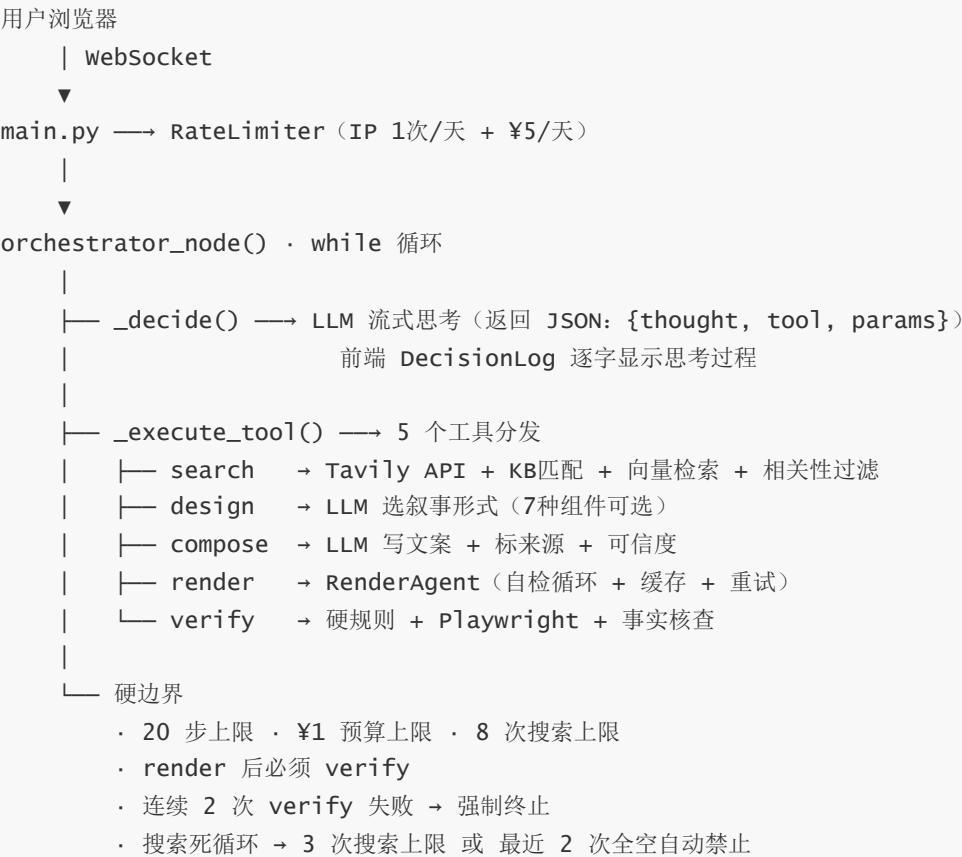
"时光像素" —— 一个 AI 原生系统。输入任意主题 → LLM 自己决定每一步做什么 → 生成交互式 HTML 页面。

- 作者：朱子钦，衡水学院 2026 届，秋招主攻 AI 应用工程岗
- GitHub：[hfujw/ai-native-workflow](#)
- 技术栈：Python 3.13 + FastAPI + DeepSeek API + React 18 + Vite 5 + Tailwind 3
- 代码量：3162 行 Python + 347 行前端 + 28 tests
- 在线演示：部署后 <https://域名.com>（待上线）

二、一句话核心理念

流程不是人写死的。一个 `async while` 循环 + LLM 自主决策，替代 LangGraph 的固定 Pipeline。

三、架构全景



四、完整数据流（从输入到输出）

1. 用户输入 "秦始皇修长城"
2. WebSocket 连接 → session_id = uuid4()[8]
3. _get_client_ip() → X-Forwarded-For 头（支持 nginx 反代）
4. RateLimiter.can_generate(ip) → 本地白名单放行 / IP 次数检查 / 日预算检查
5. KB 关键词匹配 get_event_by_keyword() → 命中 → material 填充
6. 未命中 → ChromaDB 向量检索 vector_search() → "嬴政" → "秦始皇"
7. while 循环开始
8. _decide() → LLM 流式返回 JSON {"tool": "search", "params": {"query": "秦始皇修长城"}}
9. _execute_tool("search") → Tavily API → 5 条结果 → 去重 + 去广告 + 相关性检查
10. evaluate_material() → 统计相关条目 → level: "high" (3条直接相关)
11. push → 前端 DecisionLog: "搜索完成, 5条素材"
12. _decide() → LLM: {"tool": "design"}
13. tool_design() → LLM 分析素材 → {"components": ["timeline", "cards"], "rationale": "时间轴展示修建历程..."}
14. _decide() → LLM: {"tool": "compose"}
15. tool_compose() → LLM 写文案 → {"title": "秦始皇与万里长城", "blocks": [...]}
16. _decide() → LLM: {"tool": "render"}
17. RenderAgent().run():
 - a. 查缓存 _cache_key(design, content) → 未命中
 - b. _generate() → tool_render_stream() → LLM 流式生成完整 HTML (约 10000 字符)
 - c. _self_check() → 检查 <html>/<body>/<script>/<style> 标签闭合 + 占位符
 - d. 通过 → _cache_set() → _safe_push() → 前端 iframe 展示
18. _decide() → LLM: {"tool": "verify"}
19. tool_verify():
 - a. Phase 1: 正则硬规则 - </html>存在? <script>存在? 占位符?
 - b. Phase 2: Playwright 真执行 - 启动 Chromium → page.set_content(html) → 等 800ms → 检查 JS 报错
 - c. Phase 3: 事实核查 - claims 来源覆盖率 > 50%?
20. 通过 → push("page_ready") → 前端显影动画
21. GENERATION_DURATION.observe() → Prometheus 记录端到端耗时
22. RateLimiter.record_success(ip) → 记一次试用
23. ws_manager.disconnect() → WebSocket 关闭

五、orchestrator 主循环详解

5.1 ctx 状态字典（18 个字段）

```
ctx = {  
    "session_id": "abc123",          # 会话标识 (uuid4 前 8 位)  
    "user_input": "秦始皇修长城",    # 用户原始输入  
    "_push": push_callback,          # WebSocket 消息推送回调  
    "material": [],                  # 搜索素材 + KB 匹配结果  
    "design": None,                   # tool_design 输出  
    "content": None,                 # tool_compose 输出  
    "html": "",                      # tool_render 输出  
    "visual": None,                  # 视觉参考 (预留, 未使用)  
    "steps": 0,                      # 当前步数  
    "max_steps": 20,                 # 最大步数限制  
    "search_max": 8,                 # 最大搜索次数  
    "budget_spent": 0.0,              # 已花费预算 (虚拟价)  
    "budget_total": 1.0,             # 总预算 ¥1
```

```
"passed": False,          # 上次 verify 是否通过
"issues": [],              # verify 发现的问题列表
"tool_history": [],        # 每步的工具调用记录
"cost_records": [],        # LLM Token 记账 (session 级别)
}
# 运行时动态字段:
# "honest_mode": bool
# "material_level": dict
# "force_verify": bool
# "force_next_tool": str
# "render_fail_count": int
# "_decide_fail_count": int
# "force_strategy_change": bool
```

5.2 循环每一步的逻辑

Step 0: 决策前的特殊路径 (3 种跳过 LLM 的场景)

场景	ctx 标志	行为
诚实模式 render 完毕	force_verify=True	跳过 _decide, 直接 verify
verify 驳回后强制回退	force_next_tool="render"	跳过 _decide, 直接执行指定工具
正常流程	无标志	调 _decide() 让 LLM 决定

Step 1: _decide() 让 LLM 决定下一步

- 构建上下文 summary: 用户输入 + 步数 + 预算 + 素材标题摘要 + 最近 3 步结果 + 最近问题
- 附加约束: 诚实模式提示 / 搜索死循环防护 / 强制换策略
- 调 chat_stream() 流式输出 → 前端 DecisionLog 逐字显示思考过程
- strip_fence() 清洗 markdown 围栏 → json.loads() 解析
- clean_thought() 截断重复开场白 ("我对XXX不太熟悉...")
- 返回 {"tool": "search", "thought": "...", "params": {...}}

Step 1.5: LLM 主动进入诚实模式

LLM 在 JSON 中加 "honest": true → orchestrator 自动把 tool 改为 render

Step 1.6: 搜索次数硬拦截

search_count >= search_max(8) 且 LLM 选了 search → 强制改为 design

Step 2: thought 推前端

防御性类型检查: isinstance(thought, dict) → 如果 LLM 把完整 JSON 塞进 thought 字段, 只取 thought.get("thought")

Step 3: 心跳 (heartbeat)

asyncio.create_task(heartbeat()) → 每 4 秒推一次 pulse, 最多 15 次 (60 秒) 。 finally: hb.cancel() 确保工具执行完后心跳停止。

Step 4: 执行工具

_execute_tool(tool_name, params, ctx) → 5 个 if-elif 分支分发

Step 5: 搜索结果评估

`evaluate_material()` 纯规则判定 → level 为 low/none 时推一条建议给前端

Step 6: 硬检查

- render 后: `complete=false` → 记录 render 失败 + 诚实模式自动 `force_verify`
- verify 后: `passed=true` → 推 `complete` → return success
`passed=false` → 强制回退 (`force_next_tool`) → `render_fail_count >= 2` → 终止

5.3 终止条件 (4 种)

1. ✔ **成功**: verify 通过 → `{status: "success", html, steps, budget}`
2. ✗ **素材不匹配**: 连续 2 次 verify 失败 → `{status: "failed", reason: "素材不匹配"}`
3. ✗ **循环耗尽**: 步数超限或预算超限 → `{status: "failed", reason: "搜了N次没找到直接素材"}`
4. ✗ **LLM 连续故障**: `_decide` 连续 3 次失败 → 触发诚实模式 + render

5.4 _decide 失败降级

```
# _decide 抛异常
↓
_decide_fail_count += 1
↓
if count >= 3: # 连续 3 次 → LLM 不可用
    honest_mode = True
    return {"tool": "render"} # 触发诚实模式终止
else:
    return {"tool": "search"} # 降级搜索 (可能解决问题)
```

六、5 个工具详解

6.1 search (Tavily + KB + 向量)

```
tool_search(query, reason, depth, existing_material):
    1. _search_tavily(query) → HTTP POST → api.tavily.com/search
        - Tavily key 为空 → 直接返回 [] (不报错)
        - 超时 15s
    2. _filter_noise() → 过滤 18 个广告关键词 (推广/促销/团购/酒店/股票...)
    3. 去重 → 跟 existing_material 比 title
    4. 相关性检查 → query 的每个词 (≥2 字符) 是否在 title+snippet 中出现
        - 全部不相关 → 返回 0 条 (不是返回噪音)
    5. 返回 {tool, query, results, count}
```

6.2 design (叙事形式选择)

```
tool_design(material, user_input):
```

7 种可选组件:

timeline (时间轴) **comparison** (对比表) **cards** (卡片集)
flowchart (流程图) **portrait** (人物画像) **datapanel** (数据面板)
encyclopedia (百科条目)

无素材 → 降级为 **["encyclopedia"]**

有素材 → LLM 分析 → {components, rationale, structure, visual_hint}

⚠ 基于自身知识设计 → 要求在 **rationale** 中标注 **"知识来源: LLM 内部知识"**

6.3 compose (文案撰写 + 来源标注)

```
tool_compose(material, design):
```

每个事实性陈述必须包含:

- **text**: 事实内容
- **source**: **"search_1"** / **"knowledge_base"** / **"llm_internal"**
- **confidence**: **"high"** / **"medium"** / **"unknown"**
- **note** (可选): **"单一来源, 史记可能夸大"**

输出: {title, subtitle, blocks: [{component, position, claims, html_hint}], fact_notes}

6.4 render (HTML 生成——RenderAgent)

```
RenderAgent().run(design, content, push, session_records):
```

Step 1: 查缓存

_cache_key = SHA256(components+rationale+structure+visual_hint + title+subtitle+blocks)

排除动态字段 (时间戳、ID) → 相同业务内容 = 相同 key

命中 → 直接返回 (0 token 花费)

Step 2: 生成 + 自检循环 (最多 2 次)

attempt 1: **_generate()** → 流式收集 HTML

↓

strip_fence() + **lstrip** 检查 DOCTYPE

↓

len < 200 → 直接返回失败

↓

_self_check():

必须存在: **<html>** + **<body>** (不是可选的)

必须闭合: **<html>/<body>/<script>/<style>** (有开必有闭)

占位符残留: {{ 不能出现

↓

通过 → 缓存 + push + return

attempt 2: **_patch_hint(design, issues)** → 在 **visual_hint** 追加修复指引

↓

_generate() 再生成 → 再自检

↓

通过 → 返回

Step 3: 两次都没过 → push 最后一次结果（残缺版）
→ `return complete:false`（让 verify 兜底）

缓存细节:

- 类级变量（`_cache: ClassVar`）——所有 RenderAgent 实例共享
- TTL 5 分钟 + 上限 50 条 + LRU 淘汰（删最老的）
- 类级变量解决第一次审查发现的“每次 new RenderAgent() 缓存永不命中”问题

推送细节:

- 生成时内部收集，不 push 前端（防流式闪烁）
- 自检通过后一次性 push——第二次审查发现重试时会推两次原始 HTML
- `_safe_push` 包裹 try/except——第三次审查发现 push 失败会中断整个请求

空内容短路:

- `_generate` 异常返回空串 → 自检的标签检查全部跳过（因为没有标签）→ 误判通过
- 第三次审查修复: `len(html) < 200` 直接返回失败

标签检查过薄:

- 最初只查 `</html>` 和 `<script>` 存在性 + 占位符
- 第三轮审查发现: LLM 输出纯文本/Markdown 时，所有检查都跳过 → 误判通过
- 修复: `<html>` 和 `<body>` 改为“必须存在”，不仅仅是“有开必有闭”

6.5 verify（三层审查）

`tool_verify(html, content):`

Phase 1: 硬规则（0ms, 纯 Python）

- `"</html>" not in html` → critical: incomplete
- `"<script" not in html.lower()` → warning: 无交互
- `"{visual_css}"` 或 `"{"` 占位符 → critical: placeholder

Phase 2: Playwright 真执行（~1.5s, 首次更慢）

- `async_playwright()` → `chromium.launch()`
- `page.on("pageerror")` → 收集 JS 报错
- `page.set_content(html)` → 等 800ms
- JS 报错 → warning

Phase 3: 事实核查（0ms）

- 遍历 `content.blocks[].claims[]`
- source 标注率 < 50% → warning: 来源不足

判决:

critical 存在 → `passed=False + rollback_target`
critical 全是 incomplete/placeholder → `rollback=render`
其他 → `rollback=compose`

七、LLM 调用层详解

7.1 三种调用模式

函数	用途	使用方
chat()	非流式，返回完整结果	design, compose (chat_json 封装)
chat_json()	temperature=0.1, 用于JSON 输出	design, compose
chat_stream()	流式，逐 chunk yield	_decide, render

7.2 chat() 内部

```
async def chat(prompt, system, model, temperature, max_tokens, session_records, label):
    messages = [system, user]

    for attempt in range(3): # MAX_RETRIES=2 → 最多 3 次
        try:
            response = await llm_breaker.call( # ← 断路器包裹
                client.chat.completions.create(
                    model="deepseek-chat",
                    temperature=temperature,
                    max_tokens=16384,
                )
            )
            content = response.choices[0].message.content

            # Token 记账 (session_records 优先, 否则全局)
            session_records.append({input_tokens, output_tokens, model})

            # Prometheus 埋点
            LLM_REQUESTS.labels(status="success", tool=label).inc()
            LLM_LATENCY.labels(tool=label).observe(duration)

            return content

        except Exception:
            wait = 2^attempt 秒 (1s → 2s → 报错)
```

7.3 chat_stream() 内部

```
async def chat_stream(prompt, system, session_records, label):
    try:
        # 先尝试带 stream_options (获取精确 token 数)
        response = await client.chat.completions.create(
            stream=True, stream_options={"include_usage": True}
        )
    except Exception:
        # DeepSeek 不支持 stream_options → 降级
```

```
response = await client.chat.completions.create(stream=True)

completion_chars = 0
async for chunk in response:
    text = chunk.choices[0].delta.content
    yield text
    completion_chars += len(text)
    if chunk.usage:
        prompt_tokens = chunk.usage.prompt_tokens
        completion_tokens = chunk.usage.completion_tokens

# 兜底：拿不到精确 token → 字符数/4 估算 (1 token ≈ 4 chars)
if prompt_tokens == 0:
    prompt_tokens = max(1, len(prompt) // 4)
if completion_tokens == 0:
    completion_tokens = max(1, completion_chars // 4)

# Token 记账 + Prometheus 埋点
```

7.4 断路器

```
class CircuitBreaker:
    state: CLOSED | OPEN | HALF_OPEN

    CLOSED (正常)
    | 连续 3 次失败
    ▼
    OPEN (熔断 30s)
    | 所有请求直接抛 CircuitOpenError, 不调 API
    | 30s 后
    ▼
    HALF_OPEN (试探)
    | 试 1 次
    |—— 成功 → CLOSED (恢复)
    |—— 失败 → OPEN (继续熔断)
```

7.5 Token 记账体系

函数	精确度	机制
chat()	精确	response.usage.prompt_tokens
chat_stream()	混合	优先 chunk.usage, 否则 字符数/4 估算

session_records 是累加式——每次 LLM 调用都 append。一个 session 结束时，get_cost_summary(records) 计算总花费。

八、知识库层

8.1 KB 关键词匹配

```
kb.py: get_event_by_keyword(text):
```

评分规则:

- alias == query: +3 (精确匹配别名)
- alias in query or query in alias: +1.5
- title == query: +3
- title in query or query in title: +1.5
- keyword in query or query in keyword: +0.5

best_score >= 1 → 返回匹配事件

否则 → None → 触发向量检索

8.2 向量语义检索

```
vector_store.py: vector_search(query, top_k=3, min_distance=1.5):
```

ChromaDB + text2vec-base-chinese (中文 embedding)

首次加载 ~400MB 从 hf-mirror.com 下载

不可用 → 返回 [], 不影响主流程

8.3 33 个示例话题

- 25 个计算机史 (verified_events.json) : ENIAC、Turing、Python、Macintosh、TCP/IP...
- 8 个八股 (verified_bagu.json) : 装饰器、闭包、协程、垃圾回收...

九、前端详解

9.1 组件树

App.jsx

├─ RevealLayer

Canvas 2D 径向渐变 mask--光标聚光灯效果

├─ SearchBubble

搜索输入框 + "免费试用 · 每日 1 次"

├─ EventTags

33 个示例话题标签云 (右上角弹出)

├─ StoryPanel

iframe 展示生成页面

|

• 流式渲染: contentDocument.write 不换 srcdoc (不频闪)

|

• 显影动画: framer-motion 从中心放大 + 亮度回归

|

• 全屏 / 最小化 / 关闭

├─ DecisionLog

AI 思考流程 (右下角悬浮卡片)

|

• 5 步进度线: 搜→定→书→绘→鉴

|

• 实时滚动气泡 (thinking / tool_result)

|

• thinking_stream: 追加到同一条, 文字持续变长

|

• StoryPanel 弹出后自动折叠

├─ FailureNotice

失败弹窗 (原因 + Demo 引导按钮)

└─ ErrorBoundary

React 错误边界

9.2 useWebSocket Hook

8 种消息类型：

- 1. `thinking` → DecisionLog 新气泡
- 2. `thinking_stream` → 同一条追加 chunk
- 3. `heartbeat` → 脉冲灯闪
- 4. `tool_result` → DecisionLog 结果摘要
- 5. `html_chunk` → iframe 流式渲染
- 6. `page_ready` → 显影动画 + `setPageHtml`
- 7. `generation_failed` → FailureNotice
- 8. 断线重连：指数退避 ×3 次 (1s → 2s → 4s)

9.3 RevealLayer（光标聚光灯）

```
Canvas 2D → createRadialGradient(cursorX, cursorY, SPOTLIGHT_R=260)
→ toDataURL() → 设为 revealDiv.maskImage
→ 两个图层叠加: base.jpg（模糊） + reveal.jpg（聚光灯区域清晰）
```

9.4 StoryPanel 流式渲染

```
// 不换 srcdoc（不频闪）
const doc = iframe.contentDocument;
doc.open();
doc.write(streamingHtml); // 每收到 html_chunk 就覆盖写
doc.close();
```

十、安全体系

层	机制	具体
输入	长度限制	<code>max_length=500</code> ，控制字符过滤
传输	HTTPS	Caddy 自动 Let's Encrypt
限流	IP + 日预算	1次/天/IP + ¥5/天全站 + 本地白名单
连接	总量 + IP	20 总上限 + 3/IP
LLM	断路器	3 次失败熔断 30s
成本	双预算	虚拟 ¥1/次 + 真实 ¥5/天
错误	友好提示	<code>_friendly_error</code> 映射，不泄露堆栈
日志	脱敏	API Key 永远不出现在日志中
iframe	sandbox	<code>allow-scripts</code> 但不 <code>allow-same-origin</code> （防 XSS）
CORS	安全	<code>allow_credentials=False</code>

十一、可观测性

11.1 Prometheus 指标 (10 个)

指标	类型	说明
llm_requests_total	Counter	LLM API 调用 (status × tool)
llm_latency_seconds	Histogram	LLM 延迟 (1s~120s × tool)
llm_tokens_total	Counter	Token 消耗 (direction × tool)
ws_connections_active	Gauge	活跃 WebSocket 连接
generations_total	Counter	生成请求 (status)
generation_duration_seconds	Histogram	端到端耗时 (10s~300s)
generation_steps	Histogram	步数分布 (1~20)
render_cache_hits_total	Counter	RenderAgent 缓存命中
render_cache_misses_total	Counter	缓存未命中
render_cache_evicted_total	Counter	缓存淘汰 (reason)

11.2 日志

- 结构化格式: key=value
- session_id 串联全链路
- TimedRotatingFileHandler: 每天午夜轮转, 保留 30 天
- 终端 INFO → stdout, 文件 DEBUG → detail.log

十二、网络层详解

12.1 WebSocket 连接管理

```
WSManager:
    connect(session_id, websocket, client_ip):
        1. 连接总数 >= 20 → 拒绝
        2. IP 连接数 >= 3 → 拒绝 (本地 IP 不限)
        3. 踢掉同 session_id 旧连接
        4. accept()

    disconnect(session_id, client_ip):
        清理连接 + 减少 IP 计数

    shutdown():
        遍历所有连接 → close(1001, "服务器维护中")
```

12.2 限流器

RateLimiter:

两层:

- IP 级: 每天 TRIALS_PER_IP 次成功生成 (失败不扣)

- 全站级: 每天 ¥DAILY_BUDGET 花费上限

本地白名单: 127.0.0.1 / ::1 / localhost → 不限

日重置: 每天午夜自动归零 (_reset_if_new_day)

当前实现: 内存 dict + asyncio.Lock

已设计预留: StateBackend 接口已就位 (state/base.py + state/memory.py)

→ 待 Phase 5 接入 Redis, 改 STATE_BACKEND=redis 一行即可

十三、错误处理体系

13.1 异常层次

AppError (根异常)

└─ ConfigError

└─ LLMError

└─ SearchError

└─ RenderError

└─ RateLimitError

配置错误 (启动时校验失败)

LLM API 通用失败

LLM 请求超时

搜索失败

HTML 生成失败

速率限制拒绝

13.2 多层降级

场景	降级路径
Tavily 搜索失败 →	返回 [], LLM 用自身知识
ChromaDB 不可用 →	跳过向量检索, 只用关键词
LLM API 超时 →	2 次重试 (指数退避) → 断路器熔断
_decide 异常 →	降级 search → 连续 3 次触发诚实模式
render 截断 →	RenderAgent 内部重试 → 2 次后让 verify 兜底
verify 不通过 →	强制回退指定工具 → 2 次后终止
design 异常 →	降级为 ["encyclopedia"]
compose 异常 →	返回空 blocks
WebSocket 断开 →	静默处理, 不崩主流程

13.3 友好错误映射

```
_friendly_error(e):  
    "timeout" → "AI 服务响应超时，请稍后重试"  
    "rate limit" → "请求过于频繁，请稍等片刻再试"  
    "auth" → "AI 服务认证失败，请联系管理员"  
    "connection" → "无法连接到 AI 服务，请检查网络后重试"  
    "json" → "AI 返回了异常响应，请重试"  
    兜底 → "生成过程中出现意外错误，请刷新页面后重试"
```

十四、部署体系

14.1 Dockerfile

```
FROM mcr.microsoft.com/playwright/python:v1.45.0-jammy # Playwright 官方镜像  
RUN pip install --no-cache-dir -r requirements.txt  
RUN playwright install chromium  
RUN useradd -m -u 1000 appuser # 非 root 用户（安全合规）  
USER appuser  
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "1"]
```

14.2 docker-compose

- Caddy (反代 + HTTPS)
- Backend (FastAPI, expose 8000)
- Redis (注释, Phase 2 启用)
- 卷挂载: logs + demos + data

14.3 Caddyfile 关键配置

- 自动 HTTPS (Let's Encrypt, 零配置)
- WebSocket 代理 (Upgrade + Connection 头透传)
- /metrics 端点 IP 白名单
- 安全头 (X-Content-Type-Options, X-Frame-Options, Referrer-Policy)
- Gzip + Zstd 压缩
- JSON 格式访问日志

14.4 CI/CD

```
.github/workflows/ci.yml:  
push/PR → main:  
  1. ruff check app/  
  2. pytest --cov --cov-fail-under=70  
  3. docker build
```

十五、数字速查表

数字	位置	含义
20	<code>max_steps</code>	orchestrator 最大循环步数
¥1	<code>budget_total</code>	单次生成预算上限
8	<code>search_max</code>	最多搜索次数
16384	<code>tool_render_max_tokens</code>	render LLM 调用 max_tokens
2048	<code>tool_decide_max_tokens</code>	_decide LLM 调用 max_tokens
¥0.15	<code>TOOL_COST["render"]</code>	render 工具虚拟价
¥0.03	<code>TOOL_COST["search"]</code>	search 工具虚拟价
¥5	<code>DAILY_BUDGET</code>	全站日预算上限
1	<code>TRIALS_PER_IP</code>	每 IP 每天试用次数
30	<code>log_retention_days</code>	日志保留天数
500	<code>input_max_length</code>	用户输入最大长度（字符）
200	<code>MIN_HTML_LENGTH</code>	HTML 最小内容检查（字节）
3	<code>circuit_failure_threshold</code>	断路器熔断阈值（连续失败次数）
30	<code>circuit_recovery_timeout</code>	断路器恢复等待（秒）
2	<code>MAX_RETRIES</code>	LLM 调用最大重试次数
120	<code>DEFAULT_TIMEOUT</code>	LLM 调用超时（秒）
50	<code>CACHE_MAX</code>	RenderAgent 缓存上限（条）
300	<code>CACHE_TTL</code>	RenderAgent 缓存过期（秒）
2	<code>render_retry</code>	RenderAgent 最大重试次数
15	<code>heartbeat_max</code>	心跳最大脉冲次数
4	<code>heartbeat_interval</code>	心跳间隔（秒）
2	<code>push_window</code>	渲染流推送时间窗口（秒）
300	<code>push_chars</code>	渲染流推送字符阈值
20	<code>max_connections</code>	WebSocket 最大连接数
3	<code>max_connections_per_ip</code>	单 IP 最大连接数
30	<code>receive_timeout</code>	WebSocket 接收输入超时（秒）
300	<code>generation_timeout</code>	单次生成超时（秒）

数字	位置	含义
1.5	<code>min_distance</code>	向量检索最小距离阈值
33	KB 话题	示例话题总数
28	tests	测试用例数
7	目录数	app/ 下一级目录数
78	files	项目总文件数
3162	LoC	Python 总行数

十六、今天（2026-08-08）的所有改动

架构重构（目录分层）

之前	之后	说明
<code>app/</code> 平铺 10+ .py	7 个目录 (<code>core/llm/network/tools/agents/knowledge/state</code>)	改搜索只开 <code>tools/search.py</code>
<code>app/tools.py</code> 410 行	5 个文件各 ~80 行	每个工具独立
<code>app/llm_client.py</code>	<code>llm/client.py</code> + <code>llm/parser.py</code> + <code>llm/circuit_breaker.py</code>	LLM 相关全在一起
<code>app/state/agent_state.py</code>	<code>agents/context.py</code>	命名不跟持久化 state/ 冲突
<code>app/reliability/</code>	删除	<code>circuit_breaker</code> 归到 <code>llm/</code>
<code>docs/</code> 15 个文件	9 个	4 份审查报告合并, 4 份旧文件删除
根目录 14 个文件	7 个	删 4 张截图 + <code>project-source-full.txt</code> + 空目录

Phase 1: RenderAgent（195 行新代码）

- 自检循环：生成 → 检查 4 对标签闭合 → 不通过修 → 重试（2 次）
- 缓存：类级变量 + TTL 5 分钟 + 上限 50 条 + SHA256 去时间戳
- `_safe_push`：WebSocket 断开不阻断返回
- 3 轮代码审查修了 15 个问题
- `orchestrator` 调用从 16 行减到 8 行（外表不变）

Bug 修复（7 处）

1. `vector_search` 接入 `orchestrator`（关键词未命中 → 语义兜底）
2. `chat_stream` 加 token 记账（`session_records` + 字符数/4 估算）
3. `stream_options` 降级（DeepSeek 不兼容时自动重试）

- 4. 渲染推送加 2 秒时间窗口 (CSS 大段不卡住)
- 5. DOCTYPE lstrip (防换行导致重复添加)
- 6. 缓存 key 排除动态字段 (时间戳 → 缓存真正可命中)
- 7. 3 处裸 except 补日志

安全加固 (5 处)

- 1. 输入长度限制 `max_length=500`
- 2. IP 连接数限制 `max_connections_per_ip=3`
- 3. 日志 30 天轮转 (TimedRotatingFileHandler)
- 4. health/live + health/ready 双探针
- 5. 幂等键中间件 (core/idempotency.py)

死代码清理 (3 处)

- 1. 删 AgentBuds.tsx (未使用的 5 个状态灯组件)
- 2. 删 send_progress / agent_progress (ws_manager + useWebSocket)
- 3. 删 reliability/ 空目录

新建文件 (12 个)

文件	说明
agents/render_agent.py	Phase 1 核心
agents/evaluate.py	素材评估 (从 orchestrator 拆出)
agents/context.py	AgentState 定义
llm/parser.py	strip_fence + clean_thought
tools/__init__.py	TOOL_COST + TOOL_MAP
tools/search.py ~ verify.py	5 个工具独立文件
core/idempotency.py	幂等键中间件
schemas/websocket.py	WebSocket Pydantic 校验
LICENSE	MIT
PRIVACY.md	隐私声明 + GDPR 清单
SECURITY.md	STRIDE 威胁建模
STRUCTURE.md	78 文件结构注解

十七、面试回答模板

"请介绍一下你的项目"

时光像素是一个 AI 原生系统。你给它一个主题——比如"秦始皇修长城"——它自己决定搜索什么、用什么形式呈现、写什么文案、生成什么 HTML、然后审查质量。整个过程不是我写死的 Pipeline，而是一个 async while 循环 + LLM 自主决策。每一步 LLM 告诉我"我打算做什么、为什么"，然后调用工具。前端实时展示 AI 的思考过程。

我刻意不用 LangGraph——5 个工具的复杂度用 while 循环最合适。但我把工具接口标准化成了 state-in/state-out，预留了迁移能力。面试官如果问"为什么不用 LangGraph"，我能从复杂度、调试体验、实际需求三个维度讲清楚。

"最有技术含量的部分是什么"

渲染 Agent。render 是 token 消耗最大的步骤——一次 16384 tokens、¥0.15。我做了一个 RenderAgent，内部有自检循环：生成完先自己检查 HTML 标签闭合，通过才推给前端，不通过就补全重试。还有缓存——相同 design+content 不重复调 LLM。外表没变（orchestrator 调用方式一模一样），内部升级——这验证了"把工具升级为 Agent 不改上层"的模式。

"怎么保证生成质量"

三层审查。第一层硬规则——正则检查 HTML 结构完整性。第二层 Playwright 真执行——启动无头 Chromium，把生成的 HTML 放进去跑，检测 JS 报错。第三层事实核查——文案阶段要求 LLM 给每个数字/年份/人名标注来源和可信度，审查阶段检查来源覆盖率。三层加起来比靠 LLM 自己评估可靠得多。

"怎么处理 LLM 失败"

三层防御。每次 API 调用有 2 次指数退避重试。断路器 3 次连续失败熔断 30 秒。_decide 连续 3 次失败触发诚实模式——直接生成一个"服务暂不可用"的页面。整个系统不会因为 LLM 挂了就崩。

"项目能上线吗"

部署方案已经就位——Caddy 自动 HTTPS + Docker Compose 一键启动。现在的策略是先把工程质量打磨到位，再部署上线。觉得可以上线的时候，一个 docker-compose up -d 就起来了。我的下一步是继续做多 Agent——Phase 1 RenderAgent 刚上线，Phase 2 是合并 design+compose 为 DesignerAgent。

"多 Agent 是什么意思"

现在的 5 个工具是纯函数——orchestrator 调它们，它们返回 dict。我正在把它们升级为独立 Agent，每个有自己的决策循环。Phase 1 完成了 RenderAgent（渲染+自检+缓存），Phase 2 是 DesignerAgent（合并设计+文案，内部可以"设计→写→不满意→换形式重设计"），Phase 3 是 ResearcherAgent（搜索自己决定搜几次、换什么词、什么时候停），Phase 4 引入消息总线让 Agent 之间直接通信，Phase 5 是 Redis 分布式。每个 Phase 外观不变——上层不知道里面升级了。